

Gruha Alankara: Interior Design Platform with AR and AI Integration

Project Description

Gruha Alankara is an **AI-powered Agentic Interior Design Platform** designed to revolutionize the way people plan, visualize, and personalize their living spaces. The platform leverages the capabilities of **autonomous AI agents, computer vision, large language models, voice intelligence, and augmented reality** to provide users with a seamless and interactive interior design experience.

Traditional interior design services often require significant time, financial investment, and expert consultation, making them inaccessible to a large section of society. Additionally, users frequently struggle to visualize how different furniture pieces, layouts, and design styles will fit within their existing spaces. Gruha Alankara addresses these challenges by offering an intelligent assistant that can analyze room images, understand user preferences, generate customized design recommendations, and suggest suitable furniture options within a specified budget.

At the core of the system lies an **Agentic AI architecture**, where a **Supervisor Agent** dynamically coordinates a team of specialized AI agents to accomplish user goals. Instead of following predefined workflows, the Supervisor interprets user requirements, creates execution plans, selects appropriate agents, validates outputs, handles retries, and delivers optimized recommendations. This enables the platform to behave like a collaborative team of interior design experts working together autonomously.

The platform incorporates a **Vision Agent** capable of analyzing uploaded room images using advanced computer vision techniques to identify room types, detect existing furniture, understand spatial constraints, analyze lighting conditions, and recognize color schemes. Based on this understanding, a **Design Agent** generates personalized interior design concepts, layout suggestions, color palettes, décor recommendations, and space optimization strategies aligned with the user's preferred style, such as Scandinavian, Modern, Minimalist, or Traditional.

To bridge the gap between design concepts and real-world implementation, a **Furniture Agent** recommends suitable furniture products by considering aesthetics, functionality, and budget constraints. A dedicated **Web Agent** gathers product information, compares prices, and identifies current market trends from multiple online furniture sources. A **Budget Agent** further assists users by estimating overall costs, generating expenditure breakdowns, and proposing optimized alternatives that align with financial limitations.

Gruha Alankara also integrates a **multilingual Voice Assistant**, enabling users to interact naturally with the system using speech. By supporting multiple languages, the platform enhances accessibility and usability across diverse user groups. Furthermore, the inclusion of **AR-based visualization capabilities** allows users to preview recommended furniture arrangements within their own spaces, improving confidence in purchasing decisions and reducing uncertainty.

Scenarios

Scenario 1: Personalized Interior Design with AI

Gruha Alankara leverages advanced artificial intelligence and computer vision technologies to generate personalized interior design recommendations based on user preferences, room characteristics, and budget constraints. The platform analyzes uploaded room images to identify furniture, lighting conditions, available space, and design opportunities. Using intelligent AI agents, it provides customized suggestions for furniture placement, color palettes, décor elements, and room layouts. This automated approach enables users to access professional-quality interior design guidance quickly and efficiently, making the design process more accessible, affordable, and user-friendly.

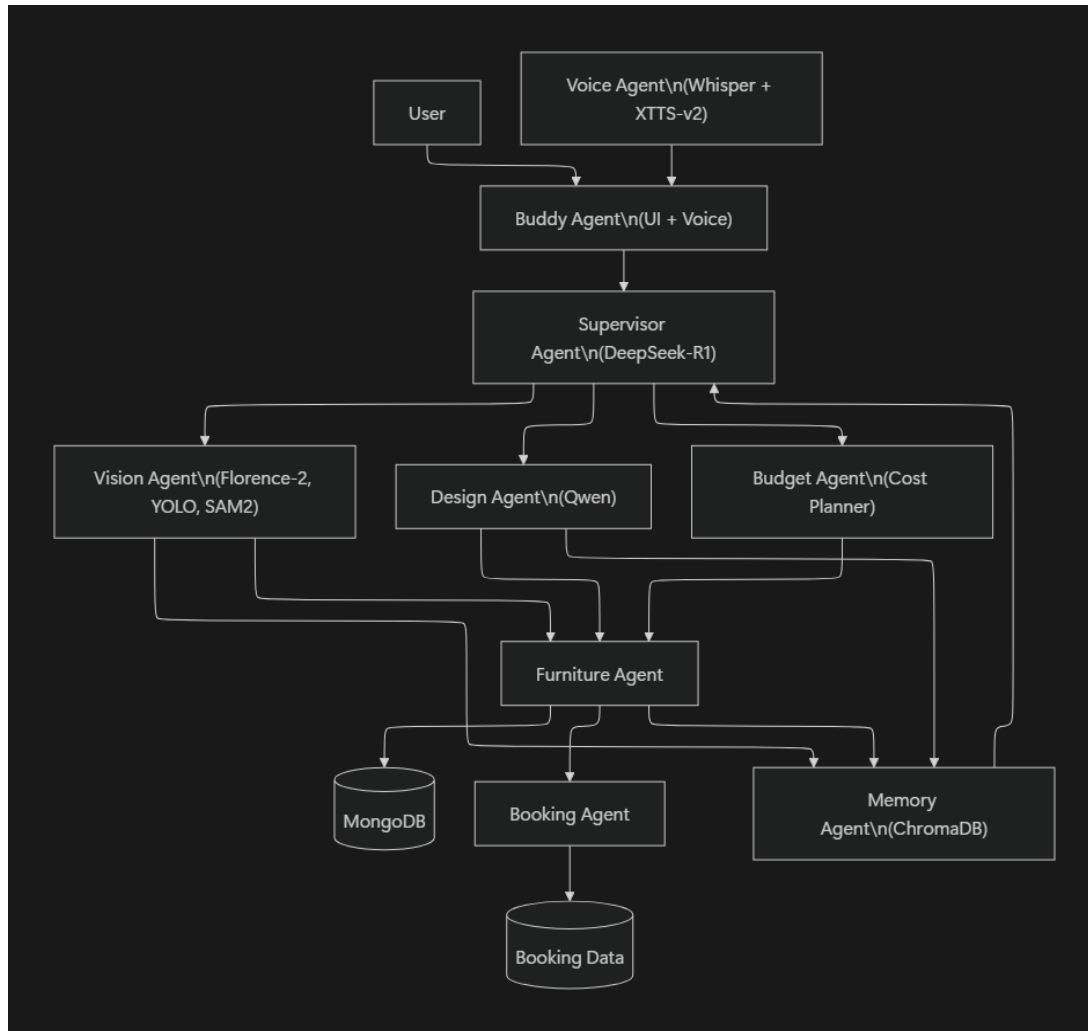
Scenario 2: Intelligent Room Visualization for Better Decision Making

Gruha Alankara enables users to preview interior design recommendations before making purchasing decisions. The platform analyzes room images and generates virtual furniture placements, allowing users to visualize furniture, color schemes, and décor elements within their space. This helps users compare design options, reduce uncertainty, and make informed decisions with greater confidence.

Scenario 3: Intelligent Furniture Recommendation and Procurement Assistance

Gruha Alankara features an AI-powered assistant that helps users select suitable furniture based on their design preferences and budget. Through text or voice interactions, the system recommends relevant products, compares available options, provides cost estimates, and assists with the booking process. This intelligent automation simplifies furniture procurement and enhances the overall interior design experience.

Technical Architecture:



System Components:

The Gruha Alankara architecture consists of three major layers: the Presentation Layer, Agentic AI Layer, and Data Management Layer. The Presentation Layer provides interfaces for room analysis, design generation, furniture recommendations, and voice interactions. The Agentic AI Layer contains the Supervisor Agent and specialized agents responsible for room analysis, design planning, budgeting, and booking assistance. The Data Management Layer utilizes MongoDB, ChromaDB, and Redis for application data, memory storage, and caching services.

Database Schema (ER Diagram):

The database consists of multiple collections including Users, Designs, Furniture, Bookings, Chat History, and Agent Memory. User information, design records, furniture recommendations, and booking details are stored in MongoDB, while personalized preferences and conversation history are maintained in ChromaDB. These collections are interconnected to support user management, design generation, recommendation systems, and booking operations.

Pre-requisites

Technical Requirements

- Python 3.10 or higher installed on the system
- Basic understanding of Flask, REST APIs, and React development
- Knowledge of MongoDB, database operations, and API integration
- Familiarity with AI/ML concepts, LangGraph, and agent-based systems
- Understanding of computer vision, voice processing, and web technologies

Development Tools

- Visual Studio Code or any modern IDE for development and debugging
- Git and GitHub for version control and project management
- Modern web browser for testing UI, camera, and voice features
- Python virtual environment manager (venv or conda)
- MongoDB Compass and Postman for database and API testing

Python Libraries

The project utilizes Flask for backend development, PyMongo for MongoDB integration, and LangGraph for agent orchestration. AI and vision functionalities are implemented using Transformers, PyTorch, Florence-2, YOLO, and SAM2 models. Additional libraries such as ChromaDB, Faster Whisper, XTTS-v2, Pillow, and OpenCV are used for memory management, voice processing, and image analysis.

Project Workflow

Phase 1: Project Initialization

- Define the objectives, scope, and core features of the Gruha Alankara platform.
- Set up the development environment and install all required dependencies.
- Create the project structure for frontend, backend, agents, and database modules.
- Plan the integration of AI models, agentic workflows, voice, and vision systems.

Phase 2: Database Setup

- Configure MongoDB and ChromaDB for application data and memory storage.
- Design collections for users, designs, furniture, bookings, and chat history.
- Establish relationships between collections and implement data management workflows.
- Apply validation rules and constraints to ensure data consistency and integrity.

Phase 3: Frontend Development

- Develop responsive React interfaces for design generation, room analysis, and user interactions.
- Implement image upload, camera access, and real-time room scanning features.
- Create intuitive dashboards for design recommendations and furniture suggestions.
- Integrate interactive visualization and design preview components.

Phase 4: AI Model Integration

- Integrate vision models for room analysis, object detection, and space understanding.
- Implement AI agents for design generation, furniture recommendation, and planning.
- Configure LangGraph for dynamic agent orchestration and workflow execution.
- Enable multilingual voice interaction using speech-to-text and text-to-speech models

Phase 5: Backend Development

- Develop Flask-based REST APIs for authentication, project management, room analysis, and design generation.
- Implement secure user authentication, authorization, JWT token management, and session handling mechanisms.
- Create API endpoints for AI agents, vision models, voice services, furniture recommendations, and booking operations.
- Integrate LangGraph to enable dynamic agent orchestration, task planning, and autonomous workflow execution.
- Establish connectivity with MongoDB, ChromaDB, Redis, and external AI services for data management and memory storage..

Phase 6: Testing and Optimization

- Perform functional testing of all frontend, backend, and AI components
- Validate image analysis, voice interaction, and recommendation accuracy.
- Optimize system performance using caching and efficient resource management
- Identify and resolve integration issues across all system modules.

Phase 7: Deployment and Maintenance

- Deploy the application on a local development server for initial testing.
- Set up monitoring tools for tracking application performance and user interactions.
- Implement comprehensive error logging and debugging mechanisms.
- Create a maintenance plan for regular updates and future feature enhancements.

Prior Knowledge

1. Flask Framework

Flask is a lightweight and flexible Python web framework used for developing scalable web applications and RESTful APIs. In the Gruha Alankara project, Flask serves as the backend framework responsible for handling API requests, user authentication, agent orchestration, database interactions, and communication between the frontend and AI services. Understanding Flask routing, middleware, session management, and API development is essential for implementing the platform's core functionalities.

Reference: <https://flask.palletsprojects.com/>

2. AI Models and Transformers Library

Artificial Intelligence models play a crucial role in enabling intelligent room analysis, design generation, and conversational interactions within the Gruha Alankara platform. The project utilizes advanced language and vision models for understanding user requirements, analyzing room images, and generating personalized interior design recommendations. Familiarity with the Hugging Face Transformers library for loading pre-trained models, processing text and image inputs, and performing inference is essential for implementing AI-driven functionalities.

Reference:

<https://huggingface.co/>
<https://huggingface.co/docs/transformers/>

3. Computer Vision and Interactive Visualization

Computer vision and interactive visualization technologies play a vital role in the Gruha Alankara platform. The system utilizes advanced vision models to analyze room images, detect furniture, identify available spaces, and understand room layouts. Knowledge of image processing, object detection, segmentation, and real-time image analysis is essential for implementing intelligent room visualization and personalized interior design recommendations.

Reference:

<https://opencv.org/>
<https://huggingface.co/docs/transformers/>

4. Frontend Development (HTML5, CSS3, JavaScript)

Frontend development is essential for creating responsive and interactive user interfaces in the Gruha Alankara platform. The project utilizes React, HTML5, CSS3, and JavaScript to build dashboards, design studios, room analysis interfaces, and visualization components. Understanding component-based development, state management, API integration, user interactions, and responsive design principles is necessary for delivering an intuitive user experience.

5. Agentic AI and LangGraph Framework

Agentic AI enables intelligent systems to perform tasks autonomously through reasoning, planning, and decision-making capabilities. In Gruha Alankara, the AI-powered Buddy Assistant interacts with users through text and voice, coordinates specialized agents, and provides personalized interior design recommendations. Familiarity with LangGraph and agent-based architectures is beneficial for understanding workflow orchestration, task execution, and intelligent automation within the platform.

Reference: <https://www.langchain.com/langgraph>

6. Database Management (MongoDB and ChromaDB)

Database management is essential for storing and managing user information, design projects, furniture recommendations, booking records, and conversation history. In Gruha Alankara, MongoDB is used for application data storage, while ChromaDB manages user preferences and memory for personalized recommendations. Understanding database modeling, collection design, data relationships, and CRUD operations is necessary for efficient data management.

Reference: <https://www.mongodb.com/>

Reference: <https://www.trychroma.com/>

Milestone 1: Environment Setup and Project Initialization Activity

1.1: Create Project Structure

- Organize the project using a modular directory structure to separate frontend, backend, AI agents, database, and configuration components.
- Configure the backend project with dedicated folders for API routes, services, middleware, utility functions, and agent implementations.
- Create frontend directories for pages, reusable UI components, assets, stylesheets, and state management modules.
- Establish folders for AI models, LangGraph workflows, memory management, vision processing, and voice services.
- Configure environment files, dependency management, logging modules, and application settings for development and deployment.
- Create dedicated storage locations for uploaded room images, generated designs, temporary files, and application logs.
- Set up database directories and connections for MongoDB, ChromaDB, Redis, and caching services.
- Implement version control using Git and establish a standardized project structure for collaboration and maintainability.
- Verify that all project modules communicate correctly and follow a scalable architecture suitable for future enhancements.
- Document the project structure and development workflow to ensure consistency throughout the implementation process.

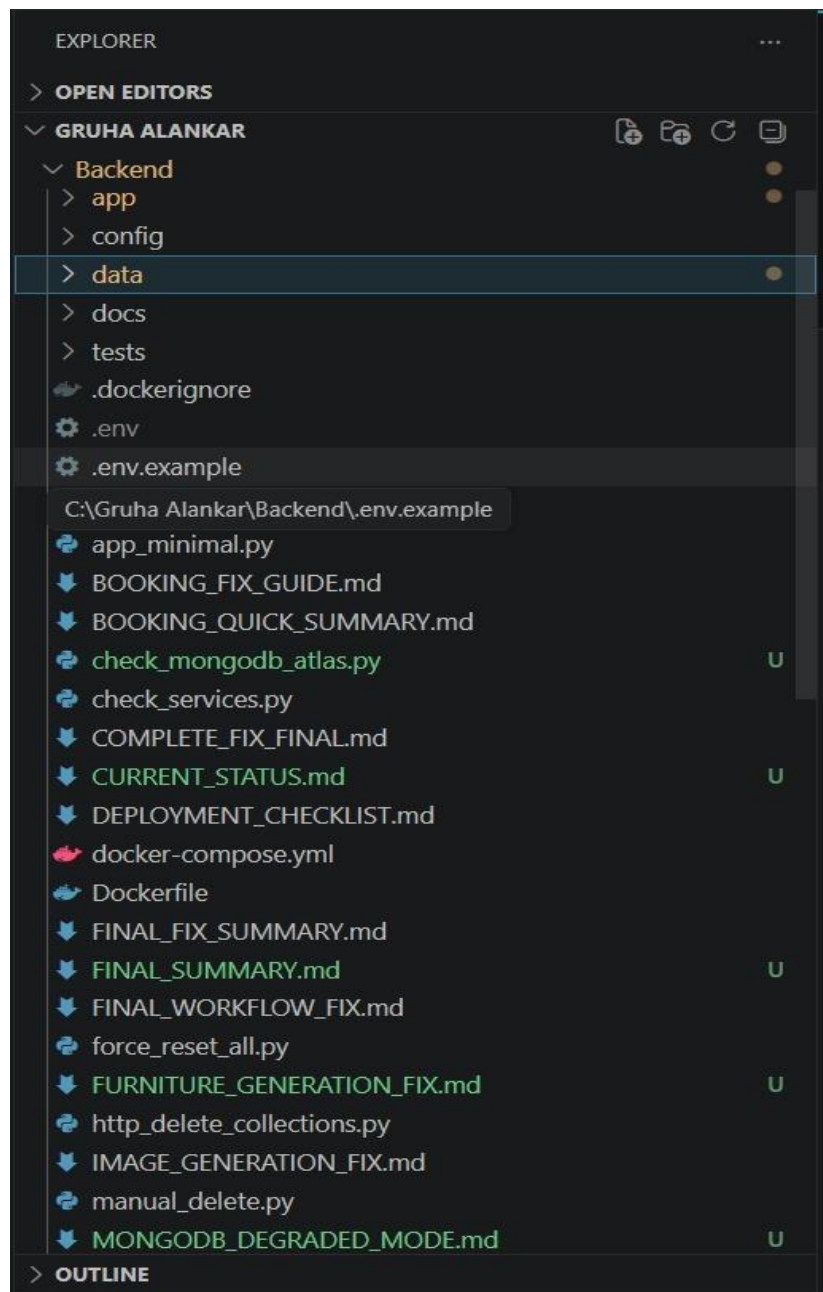
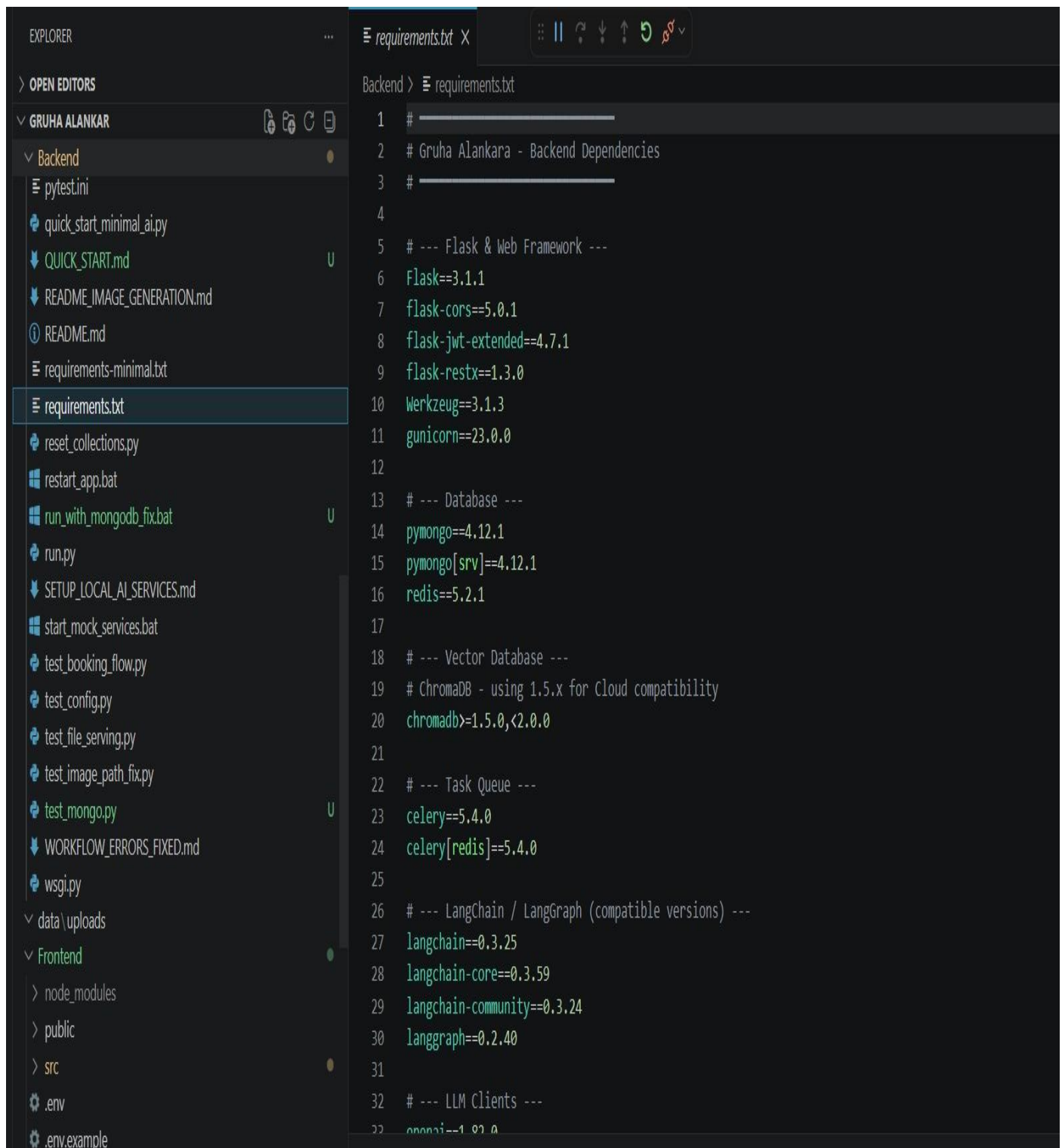


Fig: Shows the html templates and .py files of my project

Activity 1.2: Install Dependencies

- Create a Python virtual environment to isolate project dependencies and maintain a consistent development environment.
- Install all required backend packages using the `requirements.txt` file, including Flask, PyMongo, ChromaDB, LangGraph, Redis, Celery, and AI-related libraries.
- Configure frontend dependencies using Node.js and npm for React-based application development.
- Verify successful installation by testing critical libraries and ensuring all services start without errors.
- Install and configure supporting tools such as MongoDB, Redis, and ChromaDB required by the application.
- Document package versions, environment configurations, and dependency requirements for future deployment and maintenance.



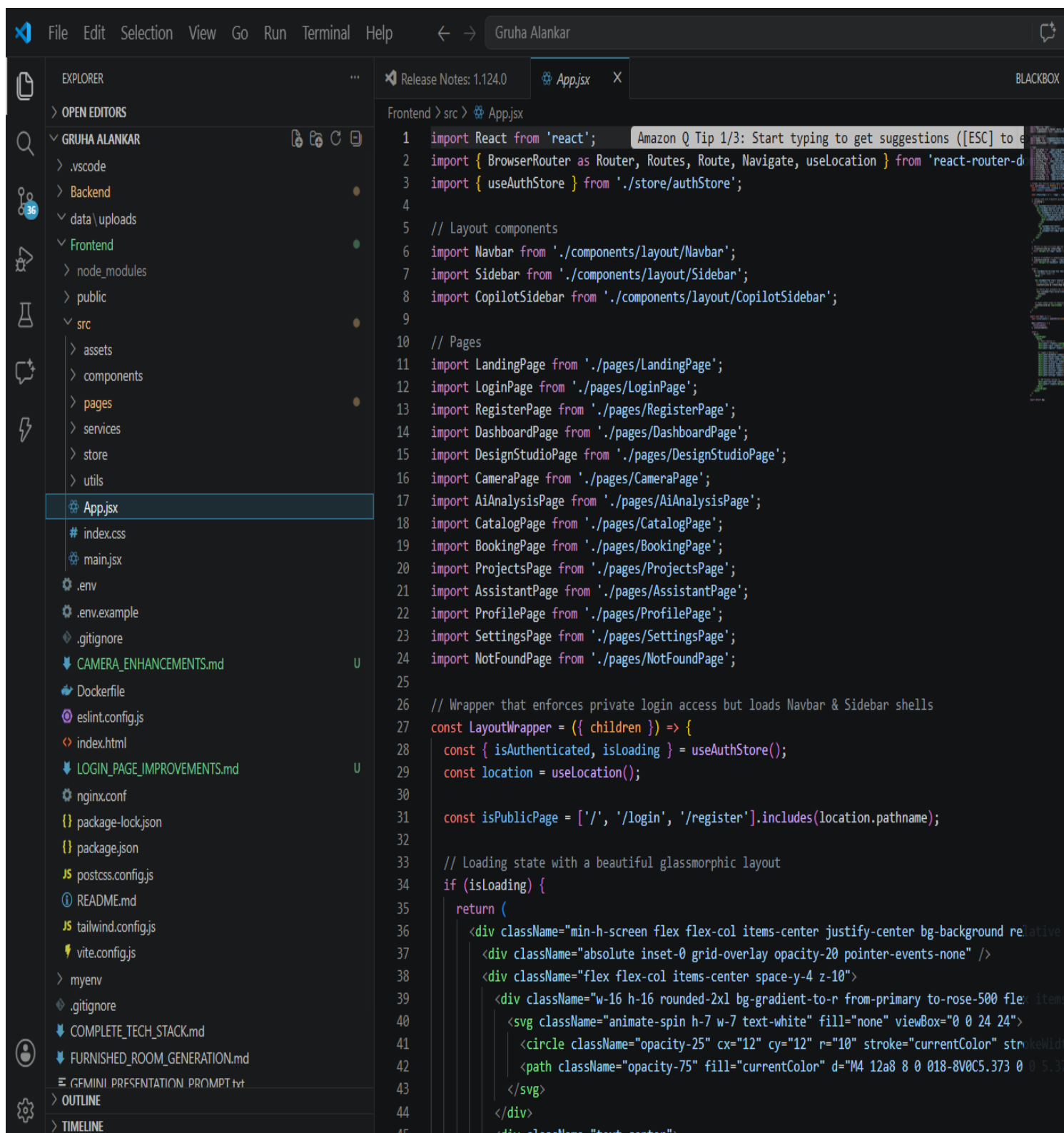
The screenshot shows the VS Code interface with the Explorer on the left and the Editor on the right. The Explorer shows the project structure for 'GRUHA ALANKAR', with the 'Backend' folder expanded. The 'requirements.txt' file is selected in the Explorer. The Editor displays the content of 'requirements.txt'.

requirements.txt

```

1 # -----
2 # Gruha Alankara - Backend Dependencies
3 # -----
4
5 # --- Flask & Web Framework ---
6 Flask==3.1.1
7 flask-cors==5.0.1
8 flask-jwt-extended==4.7.1
9 flask-restx==1.3.0
10 Werkzeug==3.1.3
11 gunicorn==23.0.0
12
13 # --- Database ---
14 pymongo==4.12.1
15 pymongo[srv]==4.12.1
16 redis==5.2.1
17
18 # --- Vector Database ---
19 # ChromaDB - using 1.5.x for Cloud compatibility
20 chromadb>=1.5.0,<2.0.0
21
22 # --- Task Queue ---
23 celery==5.4.0
24 celery[redis]==5.4.0
25
26 # --- LangChain / LangGraph (compatible versions) ---
27 langchain==0.3.25
28 langchain-core==0.3.59
29 langchain-community==0.3.24
30 langgraph==0.2.40
31
32 # --- LLM Clients ---
33 openai==1.92.0
  
```

Dependencies Or modules to import



Milestone 2: Database Configuration and Models

Activity 2.1: Configure Database Settings

- Configure environment variables and application settings for secure database connectivity.
- Set up MongoDB for storing user data, projects, furniture information, and booking records.
- Configure ChromaDB for memory management, user preferences, and conversation history.
- Establish Redis configuration for caching, session management, and background task processing.

Activity 2.2: Create Database Models

- Design collections for Users, Projects, Designs, Furniture, Bookings, and Chat History.
- Create schemas for storing user profiles, design preferences, generated layouts, and room analysis results.
- Implement collections for furniture recommendations, booking information, and order tracking.
- Design memory collections to store user preferences, conversation history, and design context.
- Establish relationships between collections to support personalized recommendations and project management.

```

1  #!/usr/bin/env python3
2  """
3  Check if MongoDB Atlas cluster is accessible and properly configured.
4  """
5
6  import requests
7  import socket
8  from urllib.parse import urlparse
9
10 def check_dns_resolution(hostname):
11     """Check if MongoDB hostname resolves."""
12     try:
13         ip = socket.gethostbyname(hostname)
14         print(f"✅ DNS Resolution: {hostname} → {ip}")
15         return True
16     except socket.gaierror:
17         print(f"❌ DNS Resolution Failed: {hostname}")
18         return False
19
20 def check_port_connectivity(hostname, port=27017):
21     """Check if port is reachable."""
22     try:
23         sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
24         sock.settimeout(5)
25         result = sock.connect_ex((hostname, port))
26         sock.close()
27
28         if result == 0:
29             print(f"✅ Port Connectivity: {hostname}:{port} is reachable")
30             return True
31         else:
32             print(f"❌ Port Connectivity: {hostname}:{port} is NOT reachable (firewall?)")
33             return False
34     except Exception as e:
35         print(f"❌ Port Check Error: {str(e)}")
36         return False
37
38 def get_public_ip():
39     """Get your public IP address."""
40     try:
41         response = requests.get('https://api.ipify.org?format=json', timeout=5)
42         ip = response.json()['ip']
43         print(f"🌐 Your Public IP: {ip}")
44         print(f"   Add this to MongoDB Atlas Network Access if not already added")
45         return ip
46     except:
47         print("⚠️ Could not determine public IP")
48         return None
49
50 def main():
51     print("\n" + "="*60)
52     print("🔍 MongoDB Atlas Connectivity Check")
53     print("="*60 + "\n")
54
55     # Check DNS Resolution
56     hostname = "mongodb-atlas.mongodb.net"
57     dns_ok = check_dns_resolution(hostname)
58
59     # Check Port Connectivity
60     port_ok = check_port_connectivity(hostname)
61
62     # Get Public IP
63     public_ip = get_public_ip()
64
65     # Summary
66     if dns_ok and port_ok:
67         print("✅ All checks passed. MongoDB Atlas is accessible.")
68     else:
69         print("❌ Some checks failed. Please check the configuration.")
70
71 if __name__ == "__main__":
72     main()

```

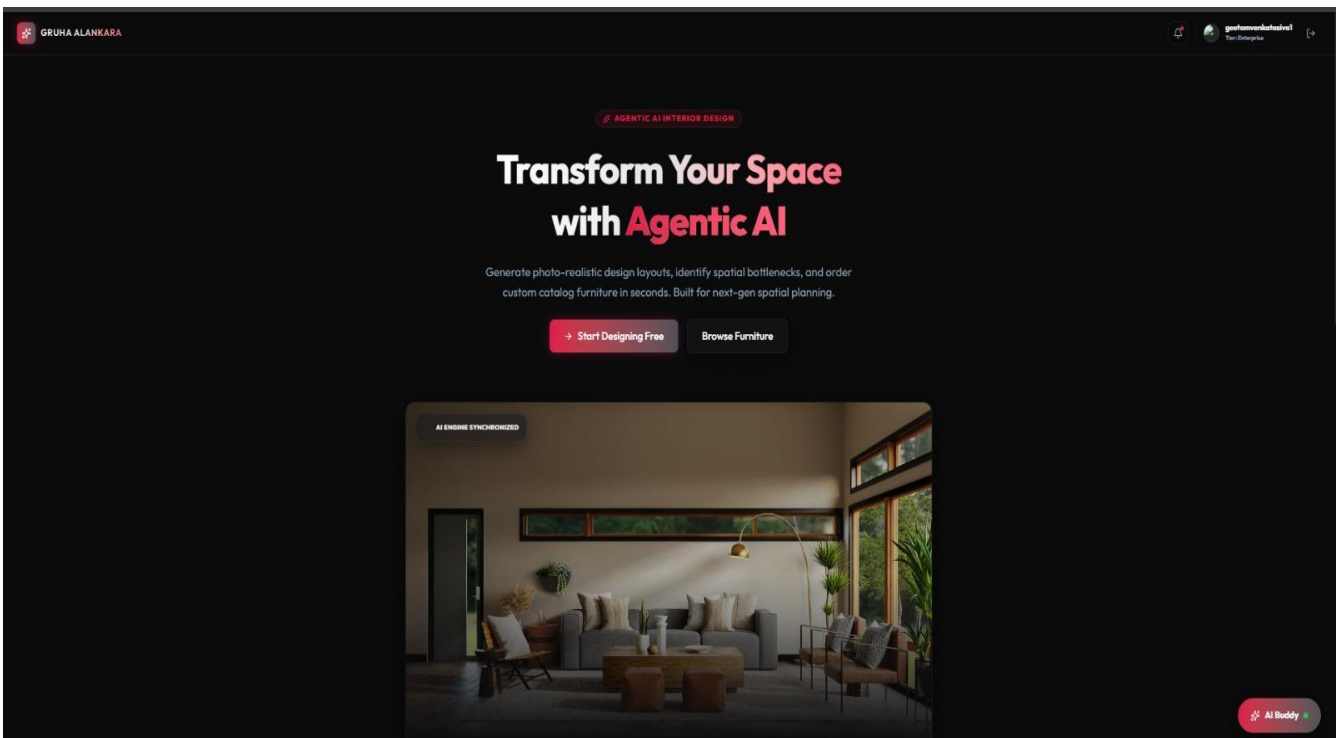
Activity 2.3: Initialize Database

- Initialize MongoDB collections and ChromaDB memory stores based on the defined application schema.
- Configure database connections within the Flask application and verify successful initialization.
- Create and populate required collections for users, projects, designs, furniture, bookings, and chat history.
- Validate collection structures, indexes, and data constraints to ensure efficient data management.
- Test database connectivity, data insertion, retrieval, update, and deletion operations.
- Verify integration between MongoDB, ChromaDB, Redis, and backend services to ensure proper system functionality.

Milestone 3: Frontend Development

Activity 3.1: Create Base Template

- Design a responsive React-based user interface that provides a consistent layout across all application modules.
- Implement a navigation bar containing platform branding, dashboard access, design studio, furniture catalog, and user profile options.
- Create reusable UI components for notifications, alerts, dialogs, and status messages to improve user experience.
- Establish a common page structure that supports dynamic content rendering and seamless navigation.
- Integrate global styling, theme configuration, and responsive layouts for desktop and mobile devices.



Activity 3.2: Create Dark-Themed CSS

- Define a dark theme color palette with consistent typography and visual styling.
- Style the navigation bar, feature cards, buttons, and interactive UI elements.
- Create reusable card components with rounded corners, spacing, and hover animations.
- Implement responsive grid layouts for displaying platform capabilities and features.
- Add smooth transitions, hover effects, and modern UI interactions for better user experience.
- Ensure responsive design across desktop, tablet, and mobile devices.

Platform Capabilities

Every feature is connected directly to our global AI copilot, orchestrating spatial geometry, logistics, and rendering.



Room Analysis

Scan lighting levels, ceiling clearances, and spatial utilization metrics instantly.



AI Design Generation

Instantly restyle rooms under Modern, Scandinavian, Industrial, or Luxury modes.



Live Camera Insights

Point your webcam to scan physical items and calculate spacing errors in real-time.



Furniture Recommendations

Browse catalog matches sized precisely to fit your room dimensions.



3D Room Rendering

Interact, zoom, and rotate furniture setups on a WebGL canvas before booking.



AI Buddy Copilot

A global companion ready to answer questions, explain designs, and translate languages.

AI Design Studio

Configure parameters, select style profiles, and generate photorealistic designs

01. ROOM BLUEPRINT



Drag & drop your room photo
or click to browse from system files

Supports JPG, PNG, WEBP files up to 10MB



Interactive Workspace Standby

Please upload a room image snapshot on the left panel, configure your target style/budget, and execute the generator.

02. CONFIGURATION SETTINGS

ROOM TYPE

Living Room

ESTIMATED BUDGET (INR)

₹ e.g. 200000

03. DESIGN STYLE PROFILE

Modern

Clean lines, geometric shapes, and functional simplicity.

Minimalist

Sparse decor, neutral tones, and maximum space...

Scandinavian

Light woods, cozy textiles, and natural light focus.

Luxury

Velvet, marble, metallic accents, and rich bold palettes.

Industrial

Exposed bricks, dark steel, concrete, and raw wood...

Contemporary

Sleek surfaces, state-of-the-art tech integration, and fluid...

Bohemian

Eclectic furniture, warm earthy tones, and vibrant patterns.

Traditional

Classic moldings, symmetrical layouts, and rich materials.

Activity 3.3: Implement Camera Access

- Implement camera access functionality for capturing room images and live room analysis.
- Configure camera settings to support image capture and real-time room scanning.
- Display live camera previews within the application interface.
- Handle camera permission requests and provide appropriate error messages when access is unavailable.
- Develop image capture functionality to process and analyze room photographs.
- Integrate image upload features to send captured images to backend AI services.
- Ensure proper camera resource management and cleanup when users exit the feature.

Milestone 4: Backend Development and AI Integration

Activity 4.1: Create Main Application

- Initialize the Flask application and configure environment settings, database connections, and required services.
- Set up MongoDB, ChromaDB, Redis, and LangGraph integrations within the application.
- Implement user authentication, authorization, and session management using JWT.
- Create API routes for user management, project handling, and design generation.
- Develop endpoints for image uploads, room analysis, and AI-powered recommendations.
- Integrate AI agents for design generation, furniture recommendations, budgeting, and booking assistance.
- Configure voice and vision services to support image analysis and multilingual interactions.
- Implement request validation, error handling, logging, and security mechanisms.
- Establish communication between frontend interfaces and backend APIs for seamless operation.
- Verify application functionality through testing and API validation

The screenshot shows a VS Code editor interface. On the left is the Explorer sidebar showing a project structure. The 'Backend' folder is expanded, showing subfolders like 'app' and 'agents'. The 'agents' folder is further expanded, showing files like 'furniture_agent.py', 'image_generation_agent.py', 'memory_agent.py', 'registry.py', 'schemas.py', 'supervisor_agent.py', 'vision_agent.py', 'voice_agent.py', and 'web_agent.py'. The 'furniture_agent.py' file is selected and its content is displayed in the main editor area.

The code in 'furniture_agent.py' is as follows:

```
1 """
2 Gruha Alankara – Furniture Agent
3
4 AI-powered product recommendation, ranking, and comparison
5 using Groq Llama-3.1-8b combined with web-scraped product data.
6 """
7
8 from __future__ import annotations
9
10 import json
11 from typing import Any, Dict, List, Optional
12
13 from app.agents.base_agent import BaseAgent
14 from app.agents.schemas import AgentResult, AgentTask, TaskStatusEnum
15 from config.constants import AgentName
16 from config.logging_config import get_logger
17 from config.settings import settings
18
19 logger = get_logger(__name__)
20
21 FURNITURE_SYSTEM_PROMPT = """You are an expert furniture consultant specializing in:
22 - Indian furniture market (Pepperfry, Urban Ladder, IKEA, Amazon India)
23 - Interior design styles and furniture matching
24 - Price-to-quality assessment
25 - Space optimization with furniture selection
26 - Budget-conscious recommendations
27
28 When ranking or recommending products, consider:
29 1. Style match with the design theme
30 2. Price-to-value ratio
31 3. Material quality and durability
32 4. Space compatibility
33 5. Customer ratings and reviews
34 6. Delivery and return policies
35
36 Always respond with structured JSON."""
37
38
39 class FurnitureAgent(BaseAgent):
40     """
41     Furniture recommendation agent powered by Groq Llama-3.1-8b.
42
43     Combines AI reasoning with scraped product data to provide
44     intelligent furniture recommendations, rankings, and comparisons.
45     """
```

EXPLORED

OPEN EDITORS

GRUHA ALANKAR

Backend

MONGODB_FIX_PYTHON313.md

openssl.cnf

preload_models.py

pytest.ini

quick_start_minimal_ai.py

QUICK_START.md

README_IMAGE_GENERATION.md

README.md

requirement: C:\Gruha Alankar\Backend\README.md

requirements.txt

reset_collections.py

restart_app.bat

run_with_mongodb_fix.bat

run.py

SETUP_LOCAL_AI_SERVICES.md

start_mock_services.bat

test_booking_flow.py

test_config.py

test_file_serving.py

test_huggingface_api.py

test_image_path_fix.py

test_mongo.py

test_pollinations.py

WORKFLOW_ERRORS_FIXED.md

wsgi.py

data\uploads

Frontend

myenv

.gitignore

COMPLETE_TECH_STACK.md

FURNISHED_ROOM_GENERATION.md

GEMINI_PRESENTATION_PROMPT.txt

INTEGRATION_GUIDE.md

README.md

run.py

Backend > run.py

```

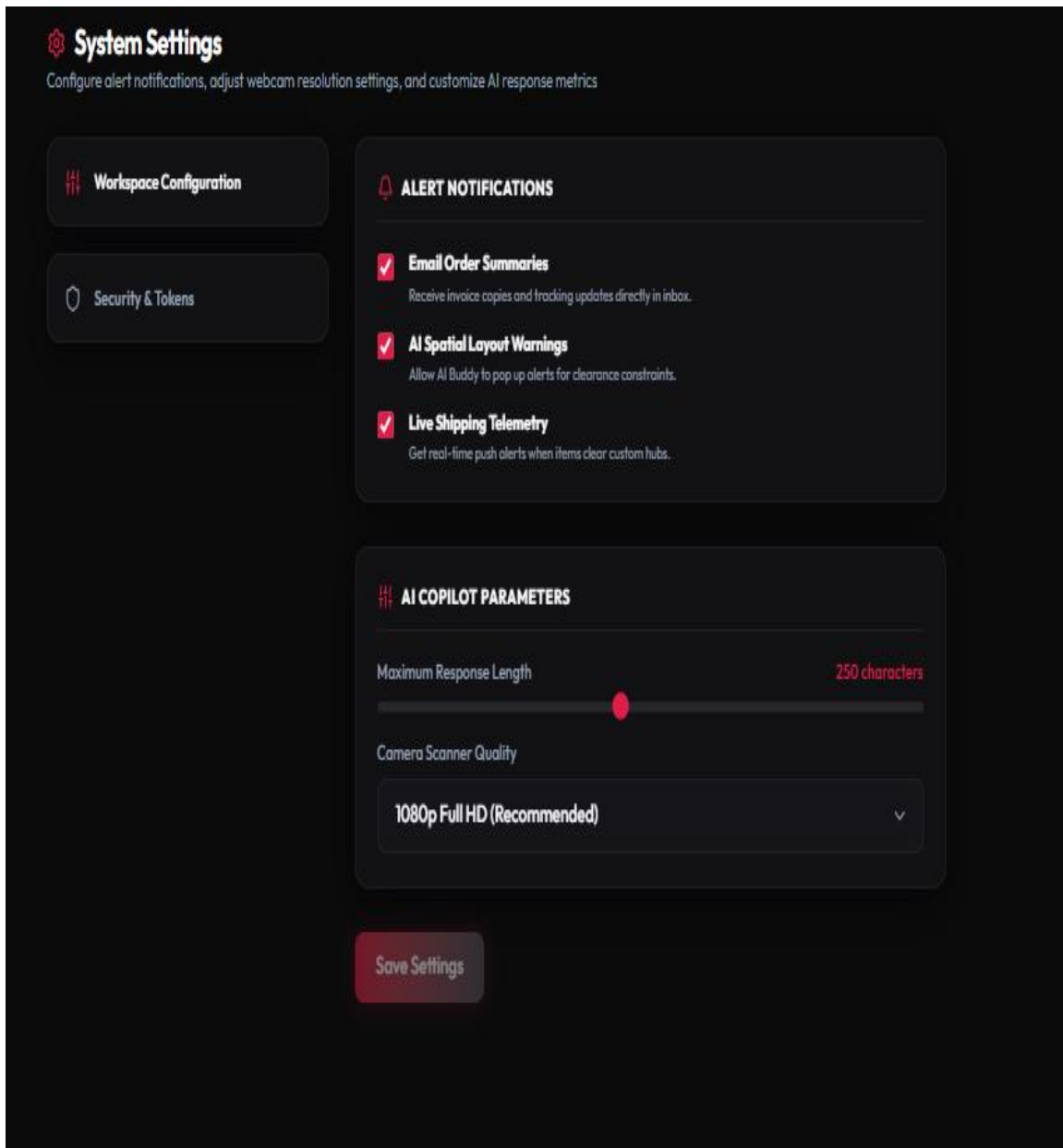
1  """
2  Gruha Alankara – Development Server Entry Point
3
4  Usage:
5      python run.py
6  """
7
8  from dotenv import load_dotenv
9
10 # Load environment variables
11 load_dotenv()
12
13 from app import create_app
14 from config.settings import settings
15
16 app = create_app()
17
18 if __name__ == "__main__":
19     app.run(
20         host=settings.server.HOST,
21         port=settings.server.PORT,
22         debug=settings.flask.FLASK_DEBUG,
23     )
24

```

Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)

Activity 4.2: Integrate AI Models

- Set up the AI design generation function that takes an image path and style theme as parameters.
- Load your pre-trained AI models or use transformer pipelines for design recommendations. Implement image preprocessing to prepare uploaded photos for model inference.
- Generate design suggestions based on the room analysis and selected style preferences.
- Format the AI output as structured text or JSON containing furniture recommendations, color schemes, and placement suggestions
- Implement error handling for AI model failures or timeouts.
- Consider implementing caching to store and reuse results for similar inputs.



System Settings
Configure alert notifications, adjust webcam resolution settings, and customize AI response metrics

Workspace Configuration

Security & Tokens

ALERT NOTIFICATIONS

- ☒ **Email Order Summaries**
Receive invoice copies and tracking updates directly in inbox.
- ☒ **AI Spatial Layout Warnings**
Allow AI Buddy to pop up alerts for clearance constraints.
- ☒ **Live Shipping Telemetry**
Get real-time push alerts when items clear custom hubs.

AI COPILOT PARAMETERS

Maximum Response Length 250 characters

Camera Scanner Quality

1080p Full HD (Recommended) ▼

Save Settings

**Copilot Full Workspace**

HIGH-PRECISION GENERATIVE AGENT

Reset Feed

AI Standby

Design Overview: Compact Serenity

The Compact Serenity design embodies a minimalist approach, focusing on clean lines, simplicity, and functionality. This style is perfect for small bedrooms, as it creates an illusion of more space while maintaining a cozy atmosphere. The design incorporates a soothing color palette and carefully selected furniture pieces to promote relaxation and tranquility.

Color Scheme

The color palette features a harmonious blend of:

- Primary Color:** Sky Blue (#87CEEB)
- Secondary Color:** Beige (#F5F5DC)
- Accent Color:** Mocha (#8B9467)
- Neutral Color:** White (#FFFFFF)

Furniture Recommendations

To create a functional and peaceful space, I recommend the following furniture pieces:

- Wall-mounted bed** (₹25,000): A space-saving bed with built-in storage drawers
- Compact dresser** (₹12,000): A slim, multi-drawer dresser for storing clothes and personal items
- Ottoman storage stool** (₹8,000): A multipurpose stool with storage for linens, out-of-season clothes, or accessories
- Floor lamp** (₹4,000): A slender, adjustable lamp for task lighting and ambiance
- Hanging shelf** (₹3,000): A floating shelf for displaying decorative items, books, or personal treasures

Decor Suggestions

To enhance the serene atmosphere, consider adding:

- Art prints or posters** (₹2,000): A set of calming, nature-inspired prints
- Potted plants** (₹1,500): A few low-maintenance plants, such as snake plants or succulents
- Textile accessories** (₹5,000): A set of throw pillows, blankets, and a rug in complementary colors

Layout Tips

To maximize the space, keep in mind:

- Keep the bed away from the door to create a sense of separation between the sleeping area and the rest of the room
- Use the ottoman storage stool as a bench or additional seating
- Consider a room divider or curtain to separate the bedroom from an adjacent area, if necessary

Budget Breakdown

The estimated total cost for this design is ₹62,000. Here's a breakdown of the costs:

- Furniture: ₹54,000 (wall-mounted bed, compact dresser, ottoman storage stool, floor lamp, and hanging shelf)
- Decor: ₹8,500 (art prints or posters, potted plants, and textile accessories)

Next Steps

Now that you have a comprehensive design plan, it's time to bring your dream bedroom to life! Here's what you can do next:

1. **Start shopping:** Look for the recommended furniture pieces and decor items within your budget.
2. **Measure your space:** Take precise measurements of your bedroom to ensure the furniture fits perfectly.
3. **Hire a professional:** If you need help with installation or assembly, consider hiring a professional to ensure everything is



Query design recommendations, layout styles, catalog pricing...



The image shows a VS Code editor interface. On the left, the 'EXPLORER (Ctrl+Shift+E)' sidebar displays a project structure for 'GRUHA ALANKAR'. The 'agents' directory is expanded, showing several Python files, with 'booking_agent.py' selected. On the right, the 'CODE EDITOR' shows the content of 'booking_agent.py'. The code includes imports for 'datetime', 'typing', and 'app.database.mongo', and defines a 'BookingAgent' class that inherits from 'BaseAgent'. The class has a docstring describing its purpose and a list of supported task types: 'create_booking', 'update_status', 'track_order', and 'list_bookings'.

```

1  """
2  Gruha Alankara - Booking Agent
3
4  Manages product bookings, order tracking, and status updates.
5  """
6
7  from __future__ import annotations
8
9  from datetime import datetime, timezone
10 from typing import Any, Dict, List
11
12 from app.agents.base_agent import BaseAgent
13 from app.agents.schemas import AgentResult, AgentTask, TaskStatusEnum
14 from app.database.mongo import (
15     insert_one,
16     find_by_id,
17     find_many,
18     update_by_id,
19 )
20 from config.constants import AgentName, BookingStatus, MongoCollection
21 from config.logging_config import get_logger
22
23 logger = get_logger(__name__)
24
25
26 class BookingAgent(BaseAgent):
27     """
28     Booking management agent.
29
30     Handles:
31     - Creating new bookings/orders
32     - Updating booking status
33     - Tracking order progress
34     """
35
36     name = AgentName.BOOKING
37     description = "Manages product bookings, order tracking, and status updates"
38     supported_task_types = [
39         "create_booking",
40         "update_status",
41         "track_order",
42         "list_bookings",
43     ]
44     estimated_latency_s = 5.0
45

```

Spatial & Lighting Diagnostics

Comprehensive AI telemetry, space efficiency grids, and illumination scores

Run Scanner



No Analysis Data Yet

Use the Live Camera Scanner to analyze your room. The diagnostics will automatically update with real data.

Open Camera Scanner

SPATIAL SCORE

Highly Efficient

92% circulation clearance rate

92%

NATURAL DAYLIGHT

Luminance: Optimal

280 Lux average ambient light

22%

FLOOR CLEARANCE

Low Obstruction

14% total floor coverage ratio

14%

ILLUMINATION DENSITY MATRIX

Calculated light levels mapping layout grids

Natural source: East



LOW LIGHT (< 50 LUX)



DAYLIGHT (> 350 LUX)

AI REASONING LOGS

DIAGNOSTIC CHECKLIST

Awaiting room analysis

Run camera scanner to generate diagnostics.

Spatial logs update automatically after running Camera Frame analysis or editing structural layouts in the 3D studio.

Milestone 5: Testing and Results

Activity 5.1: Run the Application

- Start the Flask backend server and React frontend application in development mode.
- Verify successful connections with MongoDB, ChromaDB, Redis, and other integrated services.
- Access the application through a web browser and validate that all pages load correctly.
- Check server logs and console outputs for startup errors, warnings, or configuration issues.
- Test core functionalities such as authentication, room analysis, design generation, and furniture recommendations.
- Ensure APIs, AI agents, voice services, and database operations function correctly within the application.

Activity 5.2: Test Key Features

- Test user registration, login, authentication, authorization, and session management functionalities to ensure secure access control.
- Verify room image upload, camera access permissions, image capture, and image processing features across different devices.
- Evaluate room analysis, furniture detection, space understanding, and AI-generated design recommendations for accuracy and relevance.
- Test furniture recommendation, budget estimation, product comparison, and booking assistance workflows under various scenarios.
- Validate voice interactions, multilingual support, speech recognition, and conversational AI responses for seamless user communication.
- Verify database operations, memory storage, data retrieval, and personalized recommendation mechanisms.
- Check frontend responsiveness, UI consistency, and application behavior across multiple browsers and screen sizes.

Activity 5.3: Results Documentation

- Capture screenshots of the homepage highlighting the modern user interface and key platform features.
- Document the registration and login pages demonstrating secure authentication and user management.
- Capture the room analysis and design generation interfaces showing image upload and AI-powered recommendations.
- Display examples of room diagnostics, furniture detection, and generated interior design suggestions.
- Show furniture recommendation results, budget estimation reports, and booking assistance workflows.
- Document voice interaction features, multilingual support, and conversational AI responses.
- Capture user dashboard pages displaying saved projects, design history, and personalized recommendations.
- Record the outputs of AI agents and demonstrate successful integration between frontend, backend, and database services.

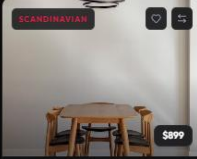
Furniture & Decor Catalog

Browse and book AI-recommended pieces configured for room space clearances

All Styles

Highest Rated

All
Sofa
Tables
Chairs
Lighting
Decor
Cabinets



SCANDINAVIAN

\$899

TABLES ★ 4.9

Nordic Oak Dining Table

Solid European White Oak dining table featuring sleek tapered legs and a smoo...

Details Book Now



BOMHEMIAN

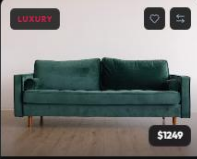
\$349

DECOR ★ 4.9

Marrakesh Tufted Wool Rug

Thick pile hand-woven organic wool rug featuring classic Moroccan diamond...

Details Book Now



LUXURY

\$1249

SOFA ★ 4.8

Aurelia Velvet Lounge Sofa

An elegant plush sofa upholstered in premium royal-purple velvet, featuring...

Details Book Now



CONTEMPORARY

\$189

LIGHTING ★ 4.8

Eclipse Pendant Drop Light

A floating celestial dome pendant light emitting a soft warm light halo, wrapped...

Details Book Now



TRADITIONAL

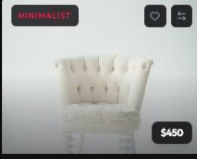
\$999

CABINETS ★ 4.8

Florence Fluted Walnut Credenza

Mid-century traditional walnut cabinet featuring beautiful fluted door faces, sof...

Details Book Now



MINIMALIST

\$450

CHAIRS ★ 4.7

Astrid Minimalist Lounge Chair

Ergonomically shaped accent chair with molded plywood frame and premium lin...

Details Book Now



INDUSTRIAL

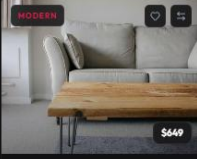
\$520

SHELVES ★ 4.7

Helix Industrial Bookshelf

A 5-tier open bookshelf featuring reclaimed ash wood boards supported b...

Details Book Now



MODERN

\$649

TABLES ★ 4.6

Calacatta Marble Coffee Table

Luxurious coffee table topped with genuine Italian Calacatta marble, resin...

Details Book Now

My Bookings & Orders

Track current shipping progress, confirm manufacturing status, and review order histories



DRAFT #6a299aef31984deb4f4f274a1

Aurelia Velvet Lounge Sofa

Wed, 10 Jun 2026 17:12:19 GMT \$7,972.82

draft Jun 10, 10:42 PM



DRAFT #6a299aef31984deb4f4f2749d

Marrakesh Tufted Wool Rug

Wed, 10 Jun 2026 17:12:15 GMT \$7910.82

draft Jun 10, 10:42 PM



DRAFT #6a2982d6f3581906c9f7cf4f

Marrakesh Tufted Wool Rug

Wed, 10 Jun 2026 15:29:26 GMT \$7910.82

draft Jun 10, 08:59 PM

Milestone 6: Future Enhancements

Performance Optimization

- Implement advanced caching mechanisms to improve AI model response times and overall system performance.
- Optimize image processing, storage management, and data retrieval operations for faster execution.
- Enhance database performance through indexing, query optimization, and efficient data management techniques.
- Improve application scalability using asynchronous processing, task queues, and load balancing strategies.
- Introduce real-time communication mechanisms to provide faster updates and a more responsive user experience.

Feature Additions

- Integrate advanced 3D visualization and virtual furniture placement capabilities for immersive design experiences.
- Add collaborative design features that allow multiple users to work on projects simultaneously.
- Enhance the AI assistant with improved reasoning, personalization, and contextual understanding capabilities.
- Expand multilingual support by introducing additional regional and international languages.
- Implement advanced recommendation systems based on user preferences, design history, and behavioral patterns.
- Add export and sharing functionality for design reports, layouts, and generated recommendations.

Security Enhancements

- Implement advanced rate limiting and access control mechanisms to protect system resources.
- Strengthen authentication and authorization using multi-factor authentication techniques.
- Enhance input validation, sanitization, and secure data handling across all application modules.
- Improve file upload security through content validation, malware detection, and access restrictions.
- Implement comprehensive monitoring, auditing, and threat detection mechanisms to enhance platform security.

Conclusion

The Gruha Alankara project successfully demonstrates the integration of Agentic AI, computer vision, voice technologies, and modern web development frameworks to create an intelligent interior design platform. The system provides personalized design recommendations through advanced room analysis, furniture detection, design generation, and budget planning capabilities. By utilizing a dynamic multi-agent architecture, the platform autonomously coordinates specialized AI agents to deliver accurate and context-aware design solutions.

The integration of vision models, conversational AI, multilingual voice support, and memory-driven personalization enhances the overall user experience and simplifies the interior design process. The modern, user-friendly interface enables users to analyze rooms, explore design ideas, receive furniture recommendations, and manage projects efficiently. Furthermore, the modular architecture ensures scalability, maintainability, and seamless integration of future enhancements.

Overall, Gruha Alankara showcases how artificial intelligence, intelligent automation, and computer vision can transform traditional interior design workflows into a smart, interactive, and highly personalized digital experience.

Project Repo Link: https://github.com/gopikiran22001/Gruha_Alankar.git

References:

Flask Documentation:<https://flask.palletsprojects.com/>

React Documentation:<https://react.dev/>

LangGraph Documentation:<https://langchain-ai.github.io/langgraph/>

LangChain Documentation:<https://python.langchain.com/>

MongoDB Documentation:<https://www.mongodb.com/docs/>

ChromaDB Documentation:<https://docs.trychroma.com/>

Hugging Face Documentation:<https://huggingface.co/docs>

Transformers Documentation:<https://huggingface.co/docs/transformers/>

Florence-2 Model:<https://huggingface.co/microsoft/Florence-2-large>

Ultralytics YOLO Documentation:<https://docs.ultralytics.com/>

Faster Whisper Repository:<https://github.com/SYSTRAN/faster-whisper>

Redis Documentation:<https://redis.io/docs/>

Celery Documentation:<https://docs.celeryq.dev/>

DeepSeek Platform:<https://platform.deepseek.com/>

Qwen Models:<https://huggingface.co/Qwen>

OpenCV Documentation:<https://docs.opencv.org/>

Three.js Documentation:<https://threejs.org/docs/>

PyTorch Documentation:<https://pytorch.org/docs/>

React Three Fiber Documentation:<https://docs.pmnd.rs/react-three-fiber>

Flask-JWT-Extended Documentation:<https://flask-jwt-extended.readthedocs.io/>